

Locally weighted & Logistic Regression

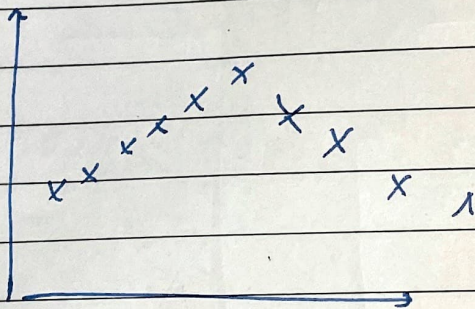
Stanford CS229 Lecture 3

Outline:

- Locally weighted regression
- Probabilistic interpretation
- Logistic regression
- Newton's method

"Parametric" learning algorithm
Fit fixed set of parameters (θ_i) to data

"Non-parametric" learning algorithm
Amount of data you need to keep grows
(linearly) with the size of the data.



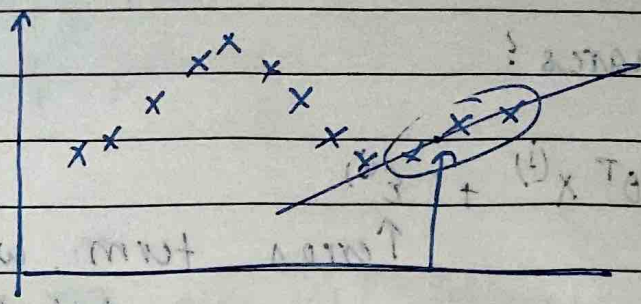
To evaluate h at certain x :

Linear Regression: Fit θ to minimize

$$\frac{1}{2} \sum_i (y^{(i)} - \theta^T x^{(i)})^2$$

return $\theta^T x$

For locally weighted regression: we see the nearby pt. & fit a straight line



Fit θ to minimize

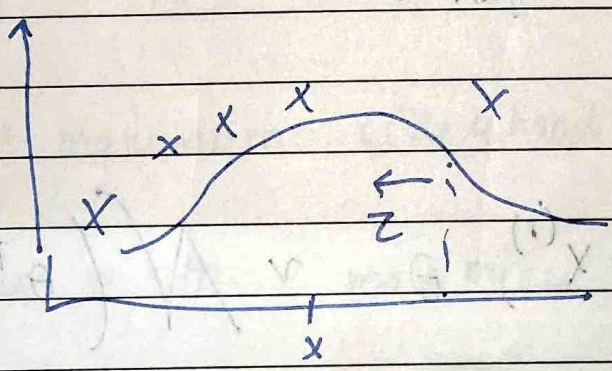
$$\sum_{i=1}^m w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$$

where $w^{(i)}$ is a "weighty" function:

$$w^{(i)} = \exp \left(- \frac{(x^{(i)} - x)^2}{2\tau^2} \right)$$

If $|x^{(i)} - x|$ is small, $w^{(i)} \approx 1$

If $|x^{(i)} - x|$ is large, $w^{(i)} \approx 0$



so modified $w^{(i)} = \exp \left(- \frac{(x^{(i)} - x)^2}{2\tau^2} \right)$

Probabilistic Interpretation

Why least squares?

Assume $y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$
 ↑ error term, which is unmodelled effects & random noise.

$$\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$$

error term is normal distributed (or gaussian distributed)

$$P(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{\epsilon^{(i)2}}{2\sigma^2}\right)$$

This implies

$$P(y^{(i)} | x^{(i)}, \theta) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

↑ mean
↑ variance

$$\text{i.e. } y^{(i)} | x^{(i)}, \theta \sim \mathcal{N}(\theta^T x^{(i)}, \sigma^2)$$

~~We made~~ - We are making assumptions that error terms are IID.

$$L(\theta) = P(\bar{y} | x; \theta)$$

"likelihood
parameter
 θ "

$$= \prod_{i=1}^m P(y^{(i)} | x^{(i)}; \theta)$$

$$= \prod_{i=1}^m \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Log likelihood

$$l(\theta) = \log L(\theta)$$

$$\sum_{i=1}^m \left[\log \frac{1}{\sqrt{2\pi}} + \log \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \right]$$

$$m \log \frac{1}{\sqrt{2\pi}} + \sum_{i=1}^m -\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}$$

MLE $\rightarrow$ maximum Likelihood Estimation

Choose θ to maximize $L(\theta)$

since this is constant, so

$$\text{minimize } \frac{1}{2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2 = J(\theta)$$

cost function

Classification

$y \in \{0, 1\}$ (binary classification).

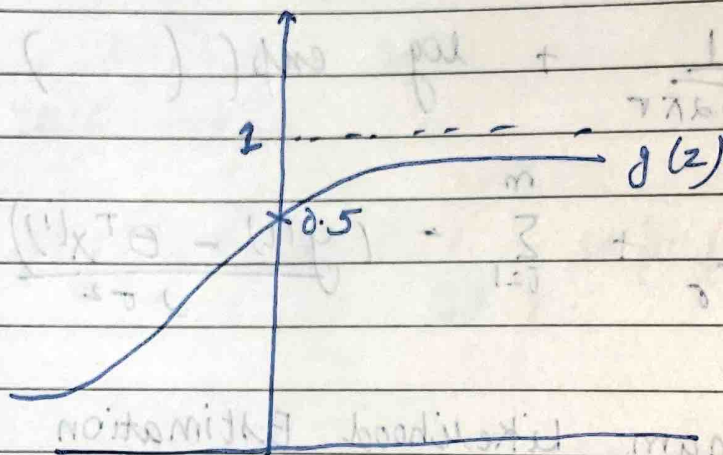
logistic regression

want $h_{\theta}(x) \in [0, 1]$

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

"sigmoid" or "logistic" function.



$$P(y=1 | x; \theta) = h_{\theta}(x)$$

$$P(y=0 | x; \theta) = 1 - h_{\theta}(x)$$

$y \in \{0, 1\}$

$$P(y | x; \theta) = h(x)^y (1 - h(x))^{1-y}$$

because both above equations can be combined.

$$L(\theta) = P(\vec{y} | x; \theta)$$

$$= \prod_{i=1}^m P(y^{(i)} | x^{(i)}; \theta)$$

$$= \prod_{i=1}^m h_{\theta}(x^{(i)})^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}$$

$$= \sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))$$

Choose θ to maximize $L(\theta)$

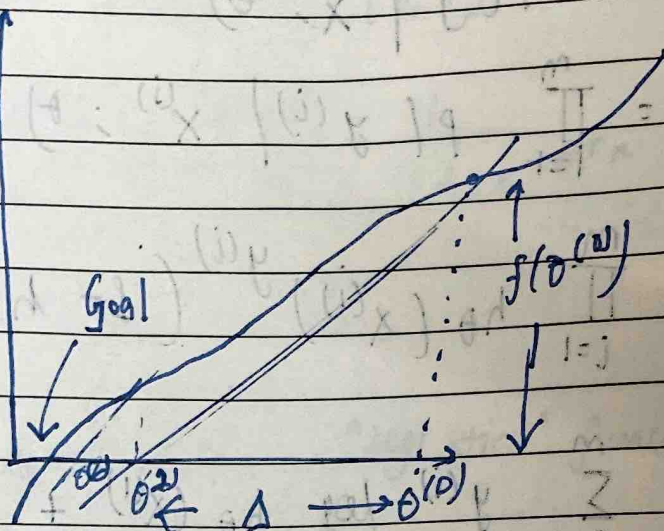
Batch Gradient Ascent

$$\theta_j := \theta_j + \alpha \frac{\partial L(\theta)}{\partial \theta_j}$$

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

Newton's Method

Have f
 we want to find θ such that $f(\theta) = 0$.
 [we want to maximize $l(\theta)$ i.e want
 $l'(\theta) = 0$]



$$\theta^{(2)} = \theta^{(1)} - \Delta$$

$$f'(\theta^0) = \frac{f(\theta^{(0)})}{\Delta}$$

$$\Delta = \frac{f(\theta^{(0)})}{f'(\theta^0)}$$

$$\theta^1 = \theta^{(0)} - \Delta$$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{f(\theta^{(t)})}{f'(\theta^{(t)})}$$

$$\text{let } f(\theta) = l'(\theta)$$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{l'(\theta^{(t)})}{l''(\theta^{(t+1)})}$$

Newton's method has a property of quadratic convergence

After 1st iteration $\rightarrow$ 0.01 error $\rightarrow$ 0.0001 error
 2nd iteration

thats why Newton's method is a fast algorithm.

~~$\theta^{(t+1)}$~~ $\approx$ when θ is a vector:
 $\theta \in \mathbb{R}^{n+1}$

$$\theta^{(t+1)} := \theta^{(t)} + H^{-1} \nabla_{\theta} l$$

where

$H =$ Hessian matrix
 $\nabla_{\theta} l$ vector of derivatives
 $\mathbb{R}^{n+1} \rightarrow \mathbb{R}^{n+1}$

$$H_{ij} = \frac{\partial^2 l}{\partial \theta_i \partial \theta_j}$$